
Should I use SAX or DOM?

Author Name : Nazmul Idris
Created On : May 22, 1999
Modified On : May 23, 1999 5:59 pm

Table of contents

Why they were both built

What is DOM?

What is SAX?

When to use DOM

When to use SAX

Conclusion

Why they were both built

SAX (Simple API for XML) and DOM (Document Object Model) were both designed to allow programmers to access their information without having to write a parser in their programming language of choice. By keeping the information in XML 1.0 format, and by using either SAX or DOM APIs your program is free to use whatever parser it wishes. This can happen because parser writers must implement the SAX and DOM APIs using their favorite programming language. SAX and DOM APIs are both available for multiple languages (Java, C++, Perl, Python, etc.).

So both SAX and DOM were created to serve the same purpose, which is giving you access to the information stored in XML documents using any programming language (and a parser for that language). However, both of them take very different approaches to giving you access to your information.

What is DOM?

DOM gives you access to the information stored in your XML document as a hierarchical object model. DOM creates a tree of nodes (based on the structure and information in your XML document) and you can access your information by interacting with this tree of nodes. The textual information in your XML document gets turned into a bunch of tree nodes. Figure 1 illustrates this.

Regardless of the kind of information in your XML document (whether it is tabular data, or a list of items, or just a document), DOM creates a tree of nodes when you create a Document object given the XML document. Thus DOM forces you to use a tree model (just like a Swing Tree-Model) to access the information in your XML document. This works out really well because XML is hierarchical in nature. This is why DOM can put all your information in a tree (even if the information is actually tabular or a simple list).

Figure 1 is overly simplistic, because in DOM, each element node actually contains a list of other nodes as its children. These children nodes might contain text values or they might be other element nodes. At first glance, it might seem unnecessary to access the value of an element node (e.g.: in “<name> Nazmul </name>”, Nazmul is the value) by looking through a list of children nodes inside of it. If each element only had one value then this would truly be unnecessary. How-

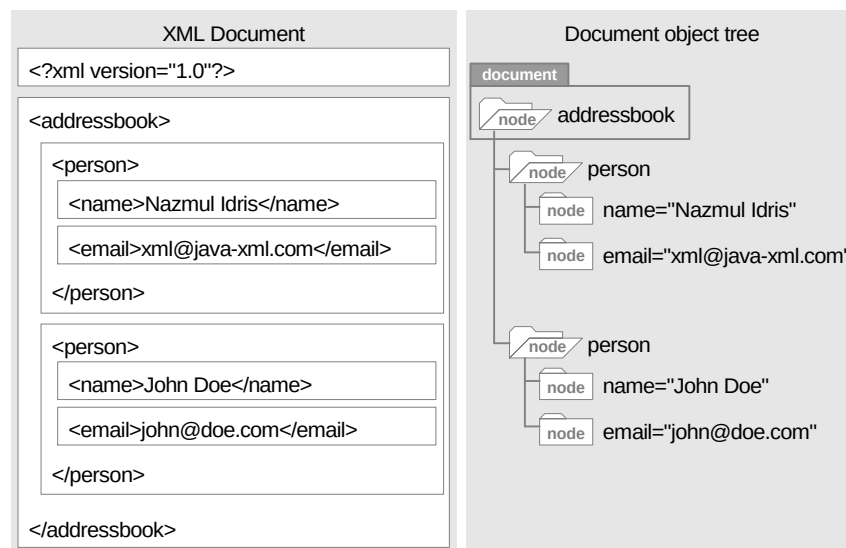


Figure 1 : DOM's tree based object model for information in an XML document

ever, elements may contain text data and other elements; this is why you have to do extra work in DOM just to get the value of an element node. Usually when pure data is contained in your XML document, it might be appropriate to “lump” all your data in one String and have DOM return that String as the value of a given element node. This does not work so well if the data stored in your XML document is a document (like a Word or Framemaker document). In documents, the sequence of elements is very important. For pure data (like a database table) the sequence of elements does not matter. So DOM preserves the sequence of the elements that it reads from XML documents, because it treats everything as if it were a document. Hence the name DOCUMENT object model.

If you plan to use DOM as the Java object model for the information stored in your XML document then you really don't need to worry about SAX. However, if you find that DOM is not a good object model to use for the information stored in your XML document then you might want to take a look at SAX. It is very natural to use SAX in cases where you have to create your own CUSTOM object models. To make matters a little more confusing, you can also create your object model(s) on top of DOM. OOP is a wonderful thing.

What is SAX?

SAX chooses to give you access to the information in your XML document, not as a tree of nodes, but as a sequence of events! You ask, how is this useful? The answer is that SAX chooses not to create a default Java object model on top of your XML document (like DOM does). This makes SAX faster, and also necessitates the following things:

- creation of your own custom object model
- creation of a class that listens to SAX events and properly creates your object model.

Note that these steps are not necessary with DOM, because DOM already creates an object model for you (which represents your information as a tree of nodes).

In the case of DOM, the parser does almost everything, read the XML document in, create a Java object model on top of it and then give you a reference to this object model (a Document object) so that you can manipulate it. SAX is not called the Simple API for XML for nothing, it is really simple. SAX doesn't expect the parser to do much, all SAX requires is that the parser should read in the XML document, and fire a bunch of events depending on what tags it encounters in the XML document. You are responsible for interpreting these events by writing an XML document handler class, which is responsible for making sense of all the tag events and creating objects in your own object model. So you have to write:

- your custom object model to "hold" all the information in your XML document into
- a document handler that listens to SAX events (which are generated by the SAX parser as its reading your XML document) and makes sense of these events to create objects in your custom object model.

SAX can be really fast at runtime if your object model is simple. In this case, it is faster than DOM, because it bypasses the creation of a tree based object model of your information. On the other hand, you do have to write a SAX document handler to interpret all the SAX events (which can be a lot of work).

What kinds of SAX events are fired by the SAX parser? These events are really very simple. SAX will fire an event for every open tag, and every close tag. It also fires events for #PCDATA and CDATA sections. Your document handler (which is a listener for these events) has to interpret these events in some meaningful way and create your custom object model based on them. Your document handler will have to interpret these events and the sequence in which these events are fired is very important. SAX also fires events for processing instructions, DTDs, comments, etc. But the idea is still the same, your handler has to interpret these events (and the sequence of the events) and make sense out of them.

When to use DOM

If your XML documents contain document data (e.g., Framemaker documents stored in XML format), then DOM is a completely natural fit for your solution. If you are creating some sort of document information management system, then you will probably have to deal with a lot of document data. An example of this is the Datachannel RIO product, which can index and organize information that comes from all kinds of document sources (like Word and Excel files). In this case, DOM is well suited to allow programs access to information stored in these documents.

However, if you are dealing mostly with structured data (the equivalent of serialized Java objects in XML) DOM is not the best choice. That is when SAX might be a better fit.

When to use SAX

If the information stored in your XML documents is machine readable (and generated) data then SAX is the right API for giving your programs access to this information. Machine readable and generated data include things like:

- Java object properties stored in XML format
- queries that are formulated using some kind of text based query language (SQL, XQL, OQL)

- result sets that are generated based on queries (this might include data in relational database tables encoded into XML).

So machine generated data is information that you normally have to create data structures and classes for in Java. A simple example is the address book which contains information about persons, as shown in Figure 1. This address book XML file is not like a word processor document, rather it is a document that contains pure data, which has been encoded into text using XML.

When your data is of this kind, you have to create your own data structures and classes (object models) anyway in order to manage, manipulate and persist this data. SAX allows you to quickly create a handler class which can create instances of your object models based on the data stored in your XML documents. An example is a SAX document handler that reads an XML document that contains my address book and creates an `AddressBook` class that can be used to access this information. The first SAX tutorial shows you how to do this. The address book XML document contains person elements, which contain name and email elements. My `AddressBook` object model contains the following classes:

- `AddressBook` class, which is a container for `Person` objects
- `Person` class, which is a container for name and email `String` objects.

So my “SAX address book document handler” is responsible for turning person elements into `Person` objects, and then storing them all in an `AddressBook` object. This document handler turns the name and email elements into `String` objects.

Conclusion

The SAX document handler you write does element to object mapping. If your information is structured in a way that makes it easy to create this mapping you should use the SAX API. On the other hand, if your data is much better represented as a tree then you should use DOM.